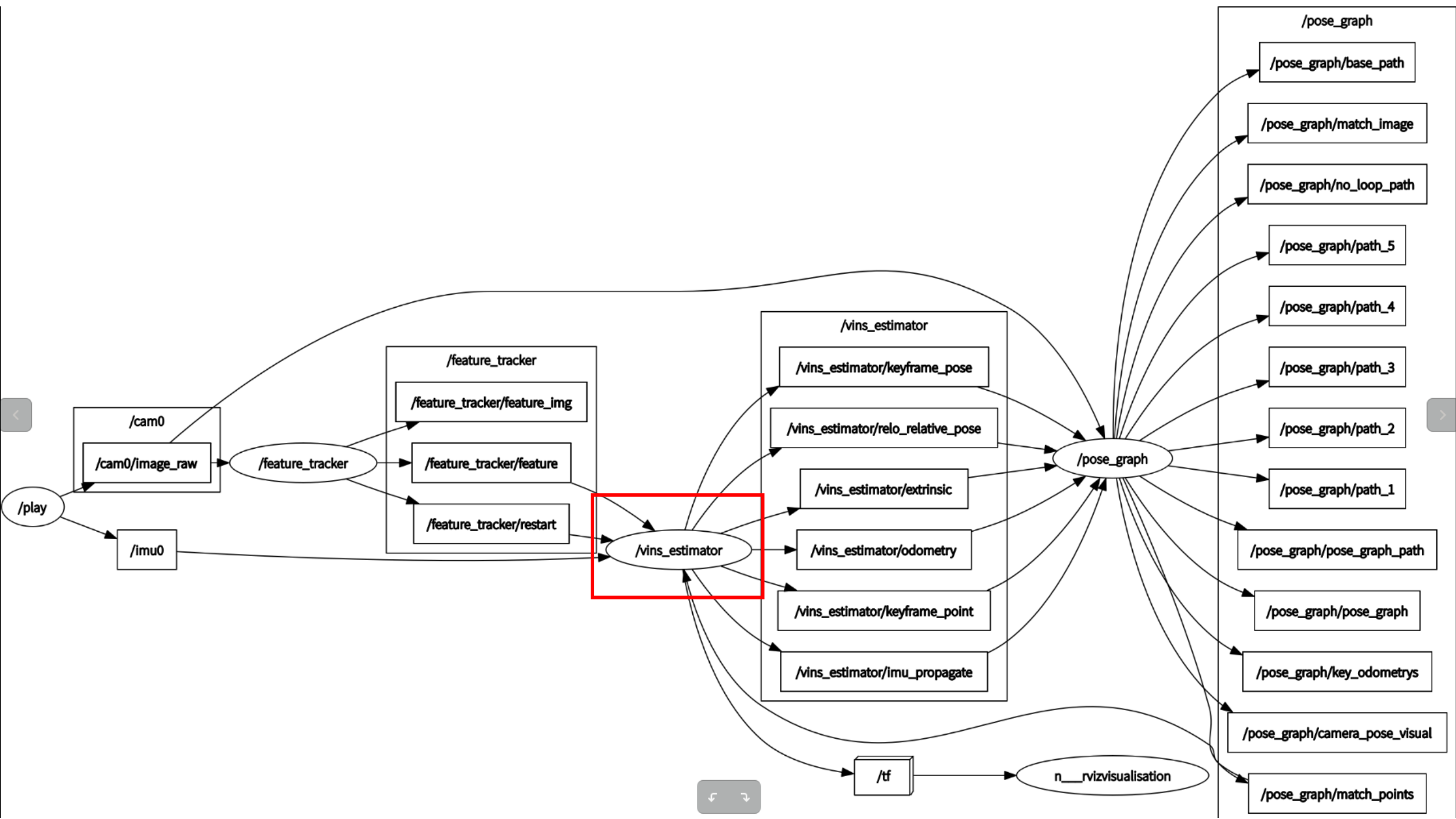
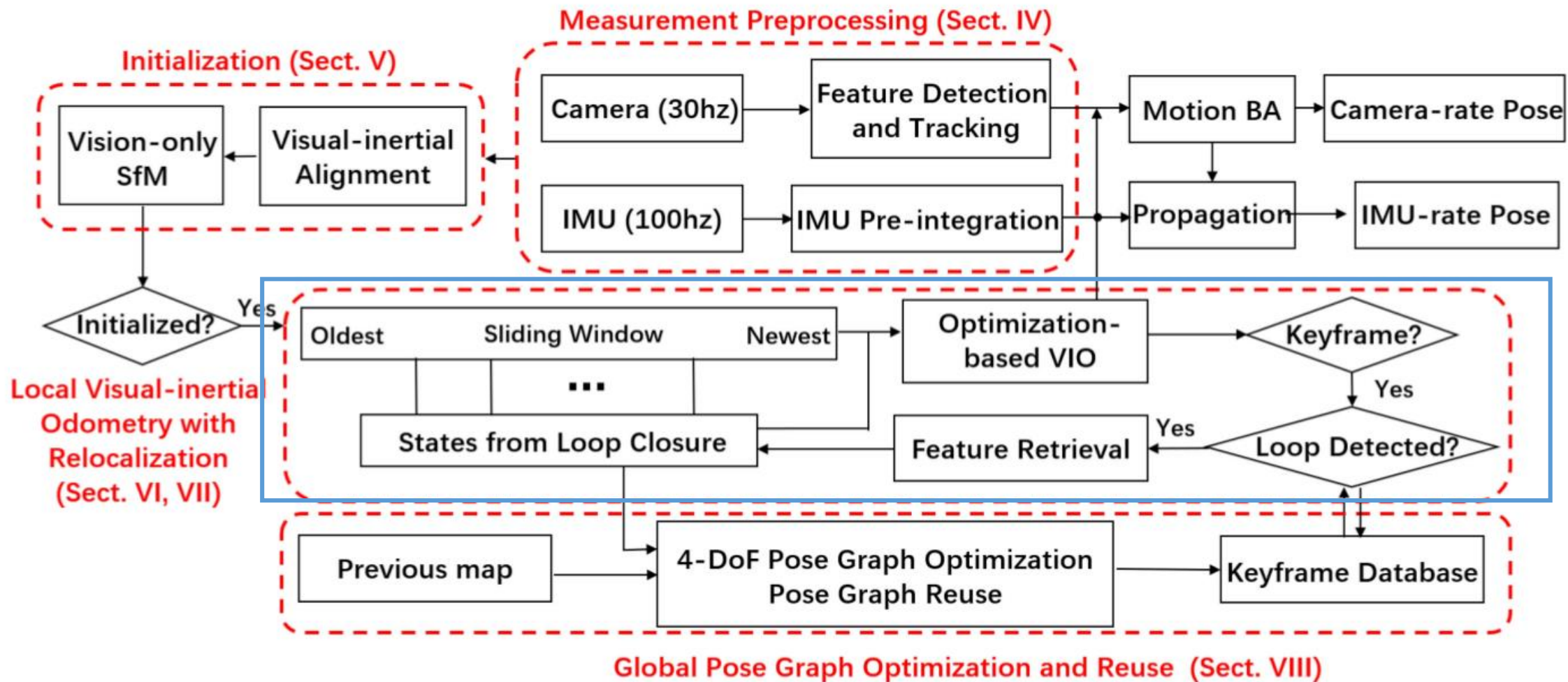


基于滑动窗口的非线性优化



课程介绍





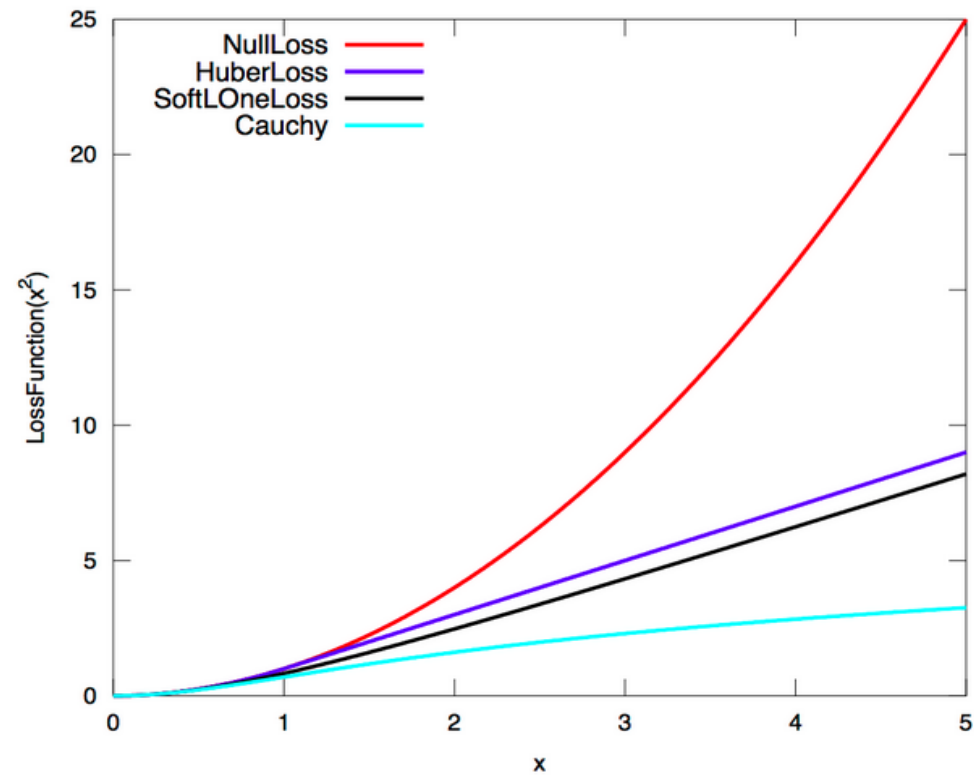
代码讲解

ceres解析求导

http://ceres-solver.org/nnls_tutorial.html

	ceres自动求导	ceres解析求导
残差、雅克比是否自己计算？	只需要定义残差计算方式	既需要定义残差计算方式 又需要定义残差对各个参数块的雅克比矩阵
costfunction的继承方式	AutoDiffCostFunction	SizedCostFunction
如何提供残差（雅克比）	自定义一个重载了括号运算符的模板函数，用来提供残差的计算方式	重载Evaluate函数用来计算残差和雅克比

loss function



Shape of the various common loss functions.

http://ceres-solver.org/nns_modeling.html#instances

预积分约束

$$\begin{aligned}
 \mathbf{R}_w^{b_k} \mathbf{p}_{b_{k+1}}^w &= \mathbf{R}_w^{b_k} \left(\mathbf{p}_{b_k}^w + \mathbf{v}_{b_k}^w \Delta t_k - \frac{1}{2} \mathbf{g}^w \Delta t_k^2 \right) + \boldsymbol{\alpha}_{b_{k+1}}^{b_k} \\
 \mathbf{R}_w^{b_k} \mathbf{v}_{b_{k+1}}^w &= \mathbf{R}_w^{b_k} (\mathbf{v}_{b_k}^w - \mathbf{g}^w \Delta t_k) + \boldsymbol{\beta}_{b_{k+1}}^{b_k} \\
 \mathbf{q}_w^{b_k} \otimes \mathbf{q}_{b_{k+1}}^w &= \boldsymbol{\gamma}_{b_{k+1}}^{b_k}
 \end{aligned} \tag{25}$$

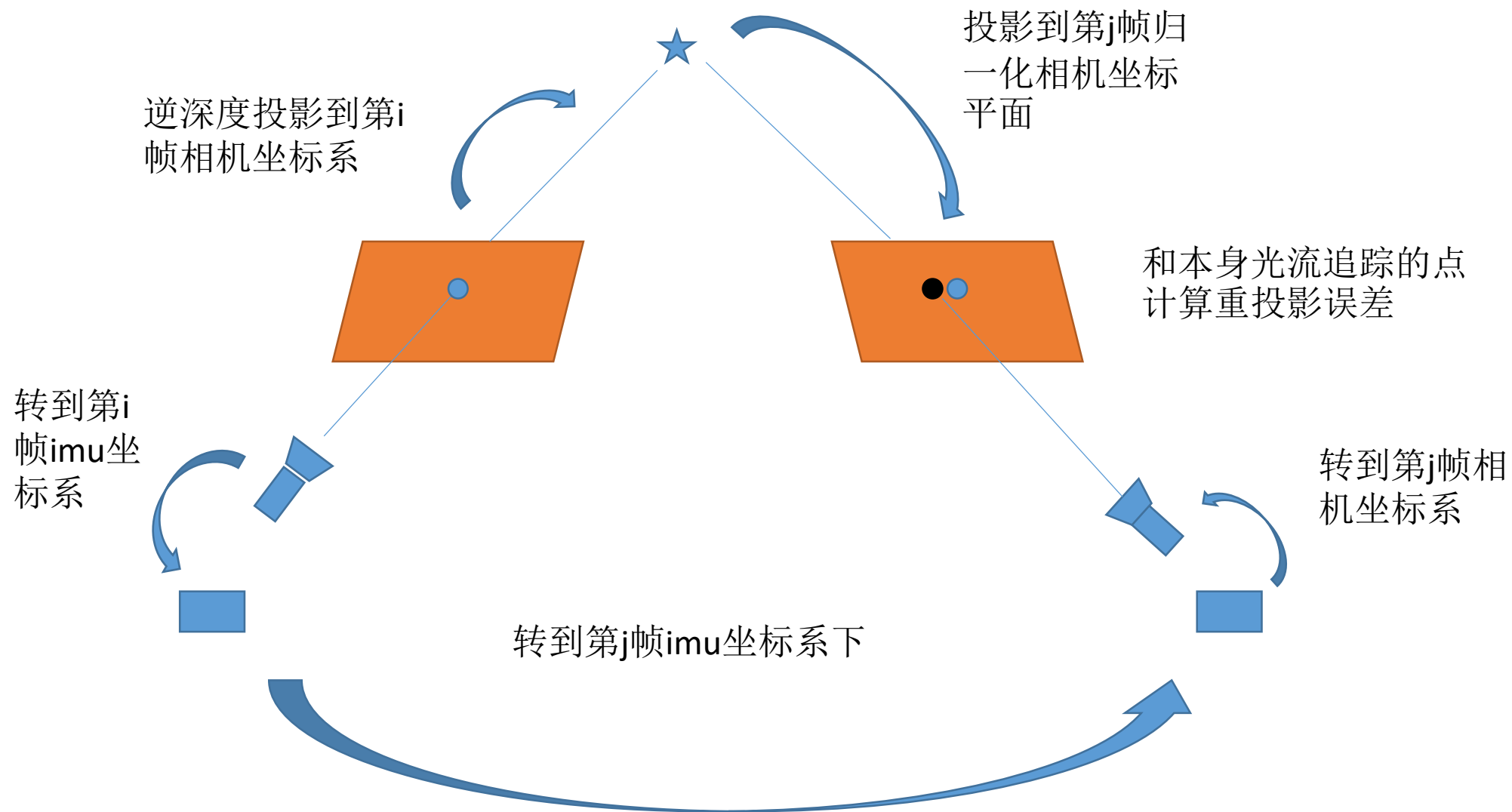
$$\mathbf{r}_{\mathcal{B}}(\hat{\mathbf{z}}_{b_{k+1}}^{b_k}, \mathcal{X}) = \begin{bmatrix} \delta \boldsymbol{\alpha}_{b_{k+1}}^{b_k} \\ \delta \boldsymbol{\beta}_{b_{k+1}}^{b_k} \\ \delta \boldsymbol{\theta}_{b_{k+1}}^{b_k} \\ \delta \mathbf{b}_a \\ \delta \mathbf{b}_g \end{bmatrix} = \begin{bmatrix} \mathbf{R}_w^{b_k} (\mathbf{p}_{b_{k+1}}^w - \mathbf{p}_{b_k}^w + \frac{1}{2} \mathbf{g}^w \Delta t_k^2 - \mathbf{v}_{b_k}^w \Delta t_k) - \hat{\boldsymbol{\alpha}}_{b_{k+1}}^{b_k} \\ \mathbf{R}_w^{b_k} (\mathbf{v}_{b_{k+1}}^w + \mathbf{g}^w \Delta t_k - \mathbf{v}_{b_k}^w) - \hat{\boldsymbol{\beta}}_{b_{k+1}}^{b_k} \\ 2 \left[\mathbf{q}_{b_k}^{w^{-1}} \otimes \mathbf{q}_{b_{k+1}}^w \otimes (\hat{\boldsymbol{\gamma}}_{b_{k+1}}^{b_k})^{-1} \right]_{xyz} \\ \mathbf{b}_{a \, b_{k+1}} - \mathbf{b}_{a \, b_k} \\ \mathbf{b}_{w \, b_{k+1}} - \mathbf{b}_{w \, b_k} \end{bmatrix}$$

$$\begin{aligned}
\boldsymbol{\alpha}_{b_{k+1}}^{b_k} &\approx \hat{\boldsymbol{\alpha}}_{b_{k+1}}^{b_k} + \mathbf{J}_{b_a}^\alpha \delta \mathbf{b}_{a_k} + \mathbf{J}_{b_w}^\alpha \delta \mathbf{b}_{w_k} \\
\boldsymbol{\beta}_{b_{k+1}}^{b_k} &\approx \hat{\boldsymbol{\beta}}_{b_{k+1}}^{b_k} + \mathbf{J}_{b_a}^\beta \delta \mathbf{b}_{a_k} + \mathbf{J}_{b_w}^\beta \delta \mathbf{b}_{w_k} \\
\boldsymbol{\gamma}_{b_{k+1}}^{b_k} &\approx \hat{\boldsymbol{\gamma}}_{b_{k+1}}^{b_k} \otimes \begin{bmatrix} 1 \\ \frac{1}{2} \mathbf{J}_{b_w}^\gamma \delta \mathbf{b}_{w_k} \end{bmatrix}
\end{aligned} \tag{33}$$

$$\delta \boldsymbol{\alpha}_{b_k+1}^{b_k} = \mathbf{R}_w^{b_k} (\mathbf{p}_{b_k+1}^w - \mathbf{p}_{b_k}^w + \frac{1}{2} \mathbf{g}^w \Delta t_k^2 - \mathbf{v}_{b_k}^w \Delta t_k) - \hat{\boldsymbol{\alpha}}_{b_k+1}^{b_k}$$

$$\delta \boldsymbol{\beta}_{b_k+1}^{b_k} = \mathbf{R}_w^{b_k} (\mathbf{v}_{b_k+1}^w + \mathbf{g}^w \Delta t_k - \mathbf{v}_{b_k}^w) - \hat{\boldsymbol{\beta}}_{b_k+1}^{b_k}$$

视觉重投影约束



转换关系用公式来描述

$$\begin{bmatrix} x_{c_j} \\ y_{c_j} \\ z_{c_j} \\ 1 \end{bmatrix} = \mathbf{T}_{bc}^{-1} \mathbf{T}_{wb_j}^{-1} \mathbf{T}_{wb_i} \mathbf{T}_{bc} \begin{bmatrix} \frac{1}{\lambda} u_{c_i} \\ \frac{1}{\lambda} v_{c_i} \\ \frac{1}{\lambda} \\ 1 \end{bmatrix}$$

第i帧归一化相机系->第j帧相机系

将旋转和平移分开之后

$$\begin{bmatrix} x_{c_j} \\ y_{c_j} \\ z_{c_j} \end{bmatrix} = \mathbf{R}_{bc}^\top \mathbf{R}_{wb_j}^\top \mathbf{R}_{wb_i} \mathbf{R}_{bc} P^{c_i} + \mathbf{R}_{bc}^\top \left(\mathbf{R}_{wb_j}^\top \left((\mathbf{R}_{wb_i} \mathbf{p}_{bc} + \mathbf{p}_{wb_i}) - \mathbf{p}_{wb_j} \right) - \mathbf{p}_{bc} \right)$$

残差的计算

$$\mathbf{r}_c = \begin{bmatrix} \frac{x_{c_j}}{z_{c_j}} - u_{c_j} \\ \frac{y_{c_j}}{z_{c_j}} - v_{c_j} \end{bmatrix}$$

通过第j帧相机系计算残差

雅克比的计算方式

$$\mathbf{r}_c = \begin{bmatrix} \frac{x_{c_j}}{z_{c_j}} - u_{c_j} \\ \frac{y_{c_j}}{z_{c_j}} - v_{c_j} \end{bmatrix} \qquad \begin{bmatrix} x_{c_j} \\ y_{c_j} \\ z_{c_j} \end{bmatrix} = \mathbf{R}_{bc}^\top \mathbf{R}_{wb_j}^\top \mathbf{R}_{wb_i} \mathbf{R}_{bc} P^{c_i} + \mathbf{R}_{bc}^\top \left(\mathbf{R}_{wb_j}^\top \left((\mathbf{R}_{wb_i} \mathbf{p}_{bc} + \mathbf{p}_{wb_i}) - \mathbf{p}_{wb_j} \right) - \mathbf{p}_{bc} \right)$$

链接求导法

$$\begin{bmatrix} x_{c_j} \\ y_{c_j} \\ z_{c_j} \end{bmatrix} = \mathbf{R}_{bc}^\top \mathbf{R}_{wb_j}^\top \mathbf{R}_{wb_i} \mathbf{R}_{bc} P^{c_i} + \mathbf{R}_{bc}^\top \left(\mathbf{R}_{wb_j}^\top \left((\mathbf{R}_{wb_i} \mathbf{p}_{bc} + \mathbf{p}_{wb_i}) - \mathbf{p}_{wb_j} \right) - \mathbf{p}_{bc} \right)$$

边缘化操作

毕业老学长

研三

研二

研一



老学长与我们同在

slam中边缘化的方式---舒尔补

$$H\delta x = b \longrightarrow \begin{bmatrix} \Lambda_a & \Lambda_b \\ \Lambda_b^T & \Lambda_c \end{bmatrix} \begin{bmatrix} \delta x_a \\ \delta x_b \end{bmatrix} = \begin{bmatrix} g_a \\ g_b \end{bmatrix}$$



$$\begin{bmatrix} I & 0 \\ -\Lambda_b^T \Lambda_a^{-1} & I \end{bmatrix} \begin{bmatrix} \Lambda_a & \Lambda_b \\ \Lambda_b^T & \Lambda_c \end{bmatrix} \begin{bmatrix} \delta x_a \\ \delta x_b \end{bmatrix} = \begin{bmatrix} I & 0 \\ -\Lambda_b^T \Lambda_a^{-1} & I \end{bmatrix} \begin{bmatrix} g_a \\ g_b \end{bmatrix}$$



$$\begin{bmatrix} \Lambda_a & \Lambda_b \\ 0 & \Lambda_c - \Lambda_b^T \Lambda_a^{-1} \Lambda_b \end{bmatrix} \begin{bmatrix} \delta x_a \\ \delta x_b \end{bmatrix} = \begin{bmatrix} g_a \\ g_b - \Lambda_b^T \Lambda_a^{-1} g_a \end{bmatrix}$$



$$(\Lambda_c - \Lambda_b^T \Lambda_a^{-1} \Lambda_b) \delta x_b = g_b - \Lambda_b^T \Lambda_a^{-1} g_a$$

H矩阵的构建过程

$$H = \sum J^T J$$

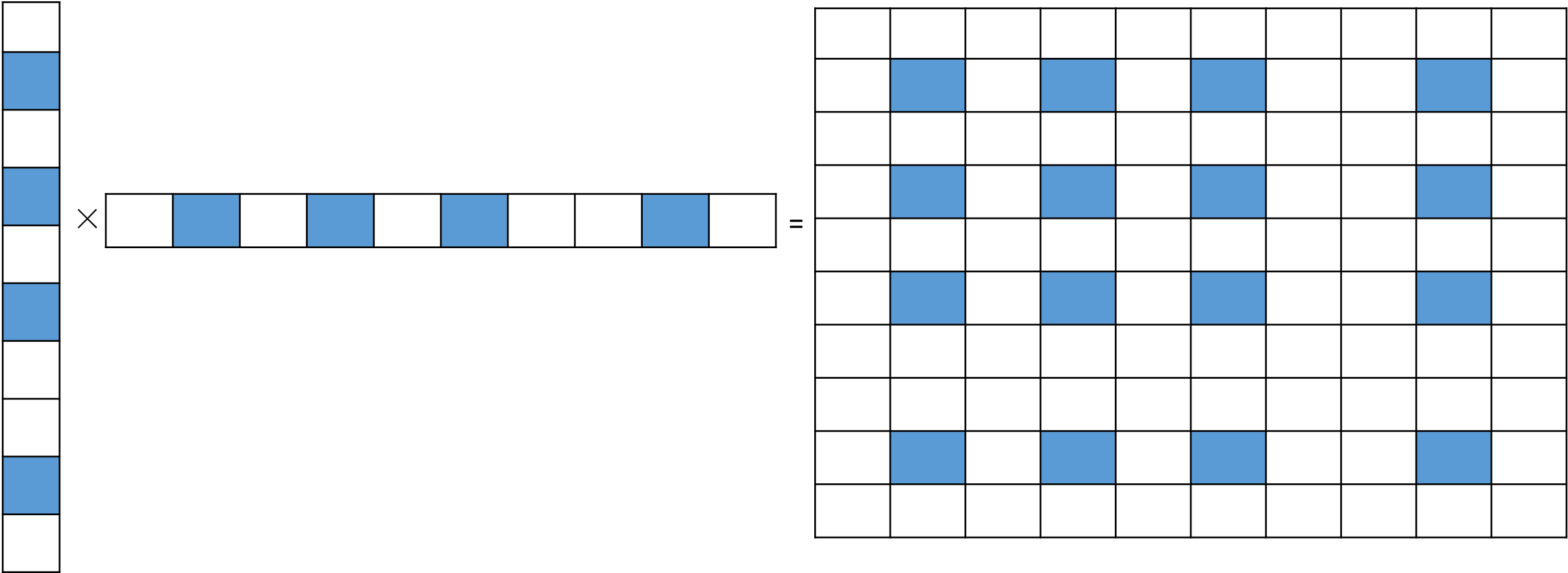
VINS中的状态向量

$$\mathcal{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots \mathbf{x}_n, \mathbf{x}_c^b, \lambda_0, \lambda_1, \dots \lambda_m]$$

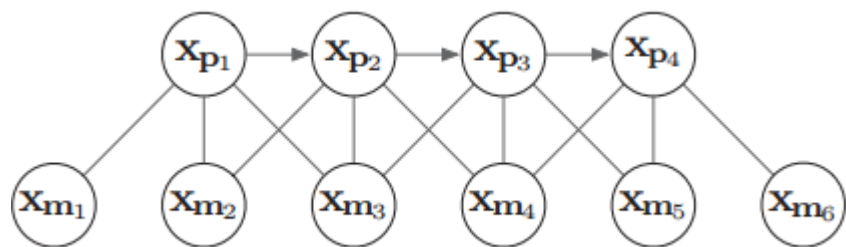
$$\mathbf{x}_k = [\mathbf{p}_{b_k}^w, \mathbf{v}_{b_k}^w, \mathbf{q}_{b_k}^w, \mathbf{b}_a, \mathbf{b}_g], k \in [0, n]$$

$$\mathbf{x}_c^b = [\mathbf{p}_c^b, \mathbf{q}_c^b]$$

以一个重投影约束为例



一个边缘化的例子



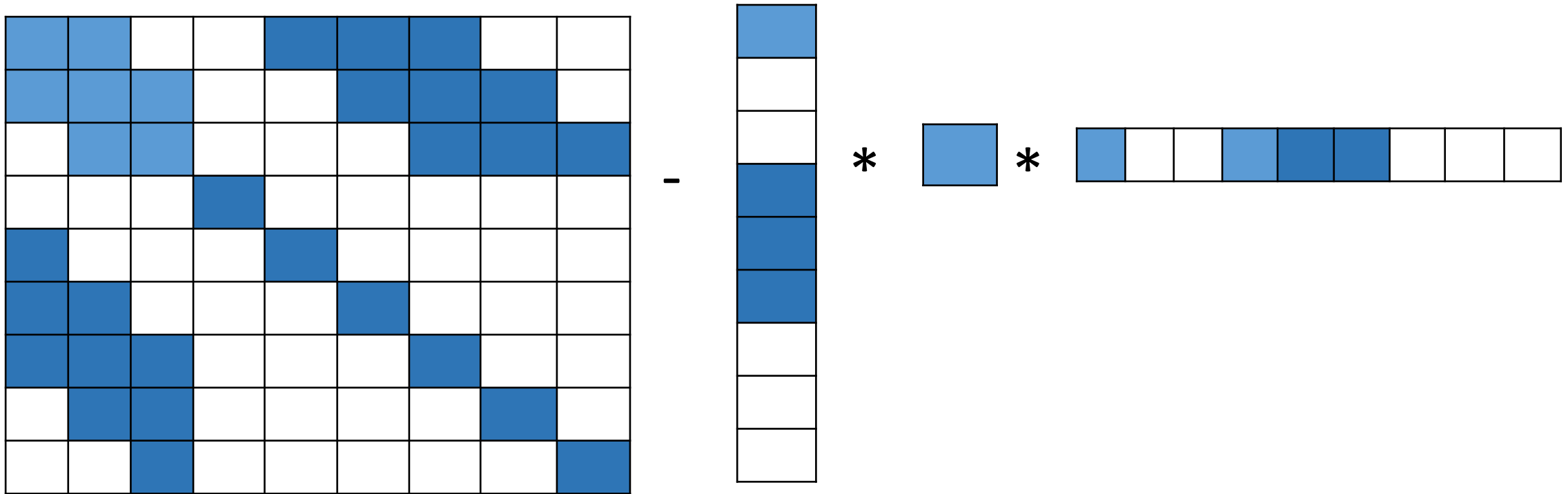
参考：

<https://blog.csdn.net/heyijia0327/article/details/52822104>

	p1	p2	p3	p4	m1	m2	m3	m4	m5	m6
p1	■	■			■	■	■			
p2	■	■	■			■	■	■		
p3		■	■	■			■	■	■	
p4			■	■				■	■	■
m1	■				■					
m2	■	■				■				
m3	■	■	■				■			
m4		■	■	■				■		
m5			■	■					■	
m6				■						■

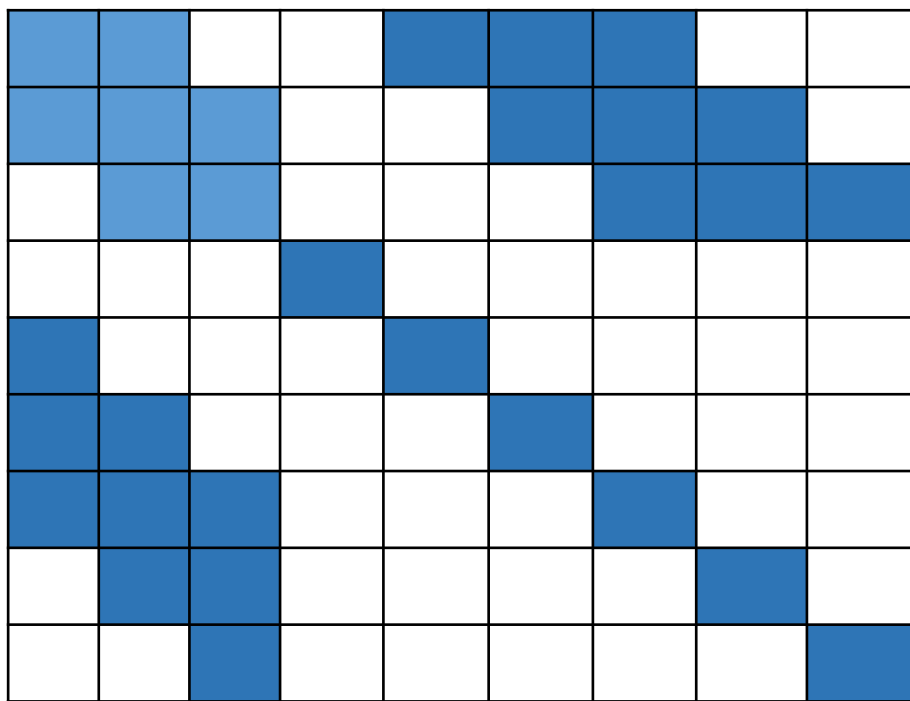
如果边缘化掉pose1会发生什么？

$$\mathbf{H} \rightarrow \Lambda_c - \Lambda_b^T \Lambda_a^{-1} \Lambda_b$$

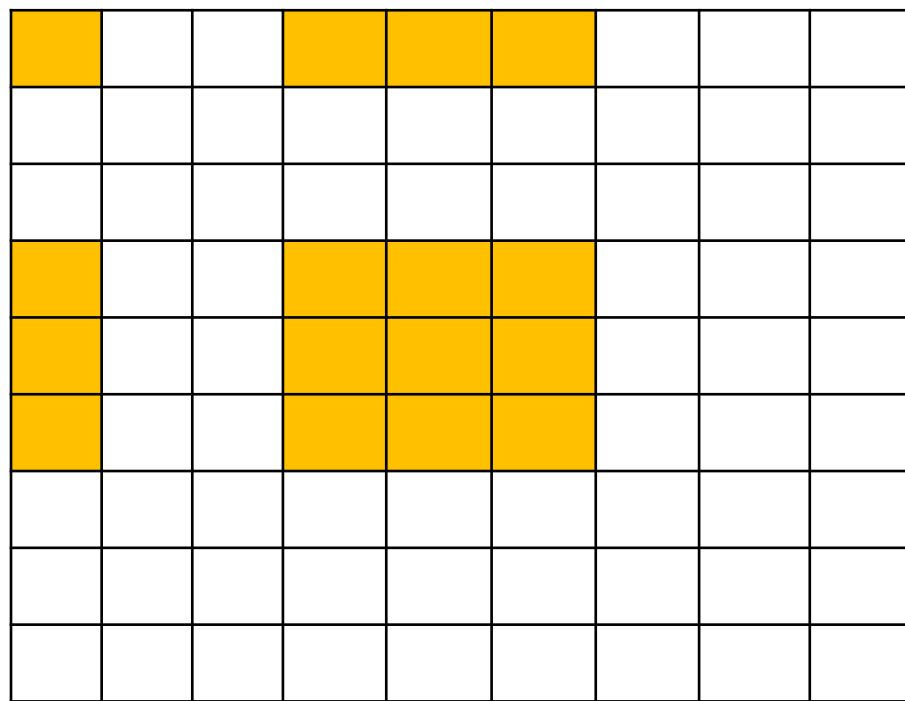


如果边缘化掉pose1会发生什么？

$$\mathbf{H} \rightarrow \Lambda_c - \Lambda_b^T \Lambda_a^{-1} \Lambda_b$$



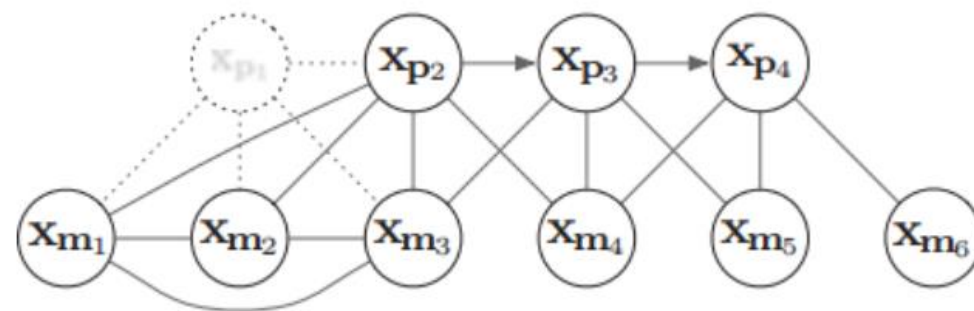
-



如果边缘化掉pose1会发生什么？

$$\mathbf{H} \rightarrow \Lambda_c - \Lambda_b^T \Lambda_a^{-1} \Lambda_b$$

	p2	p3	p4	m1	m2	m3	m4	m5	m6
p2									
p3									
p4									
m1									
m2									
m3									
m4									
m5									
m6									



边缘化带来的fill-in现象，把原先的系数矩阵变成了稠密矩阵

VINS怎么办？

边缘化掉所有的地图点，既然稠密矩阵无法避免，我们就减少矩阵的维数